

CS6023 GPU

Hello;

Kmeans clustering

By,

Prince Garg :- CS22B011

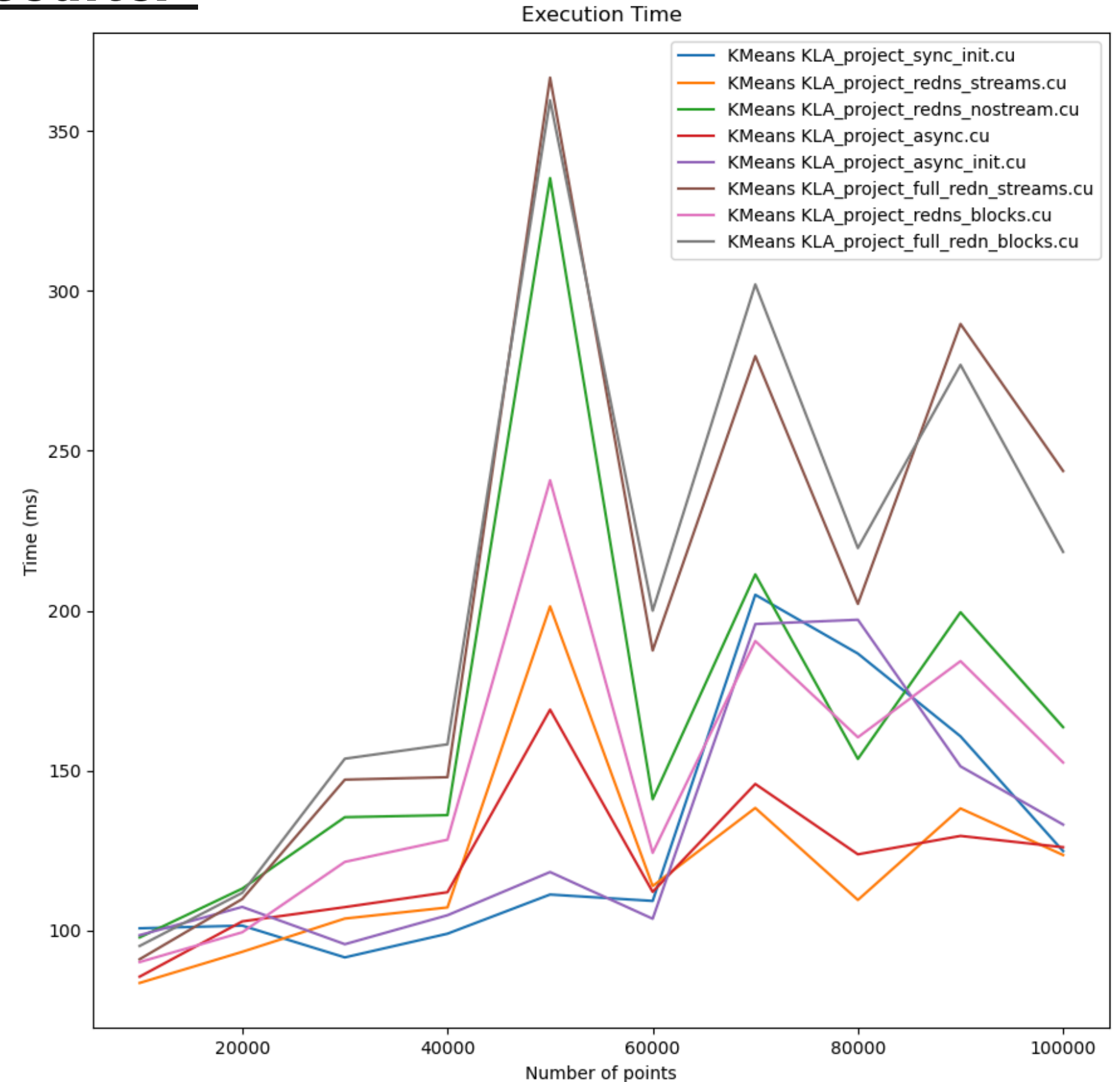
Vikash Kumar Ojha:- CS22B013

Kmeans clustering

Problem Statement:-

The dataset consists of N observations. Each observation is a d -dimensional vector. Given a set of observations $x_1, x_2, \dots, x_N$, the dataset must be partitioned into k disjoint clusters $\{S_1, S_2, \dots, S_k\}$. Suppose that m_i denotes the arithmetic mean of the data points in subset S_i . m_i is called the centroid of the cluster. The choice of clusters must minimize the sum of the square of Euclidean distances between each point in the set and the centroid. Develop a parallel solution on Nvidia GPU using CUDA and compare with that of cuML.

Results:-



Time v/s N for different implementations

Challenges Faced

We looked at the Nsight report and Ncompute report ,to know the bottle-neck and do the required optimization.

- There were a lot of atomic operations in the base code, specifically $N*d$ for calculating the average and updating the k-means. To resolve this, we implemented the reductions over the additions of the points within a group.*
- The initial choice of points mattered a lot in the computation of the final clustering, so a wise initial start point has to be chosen. To accomplish this task, we have implemented a non-probabilistic k-means parallel version to make a wise choice of the centroid before applying the algorithm.*
- To exploit full parallelism, we launched kernel with max threads for the problem statement i.e. $N*k*d$, hoping to get better timing, but that was not the case, Finally we tried with $N*d$ threads and k streams which gave us a major performance push.*
- There were several Memcopy, Malloc and Memset function which were taking a lot of time before start of actual computation, this time was reduced by using MemCpyAsync to parallelize independent malloc and other memory operation.*
- There were $k*d$ operations in the assign_centroid kernel per thread. We tried and succeeded in implementing the operations using reduction, but the time still got worse.*